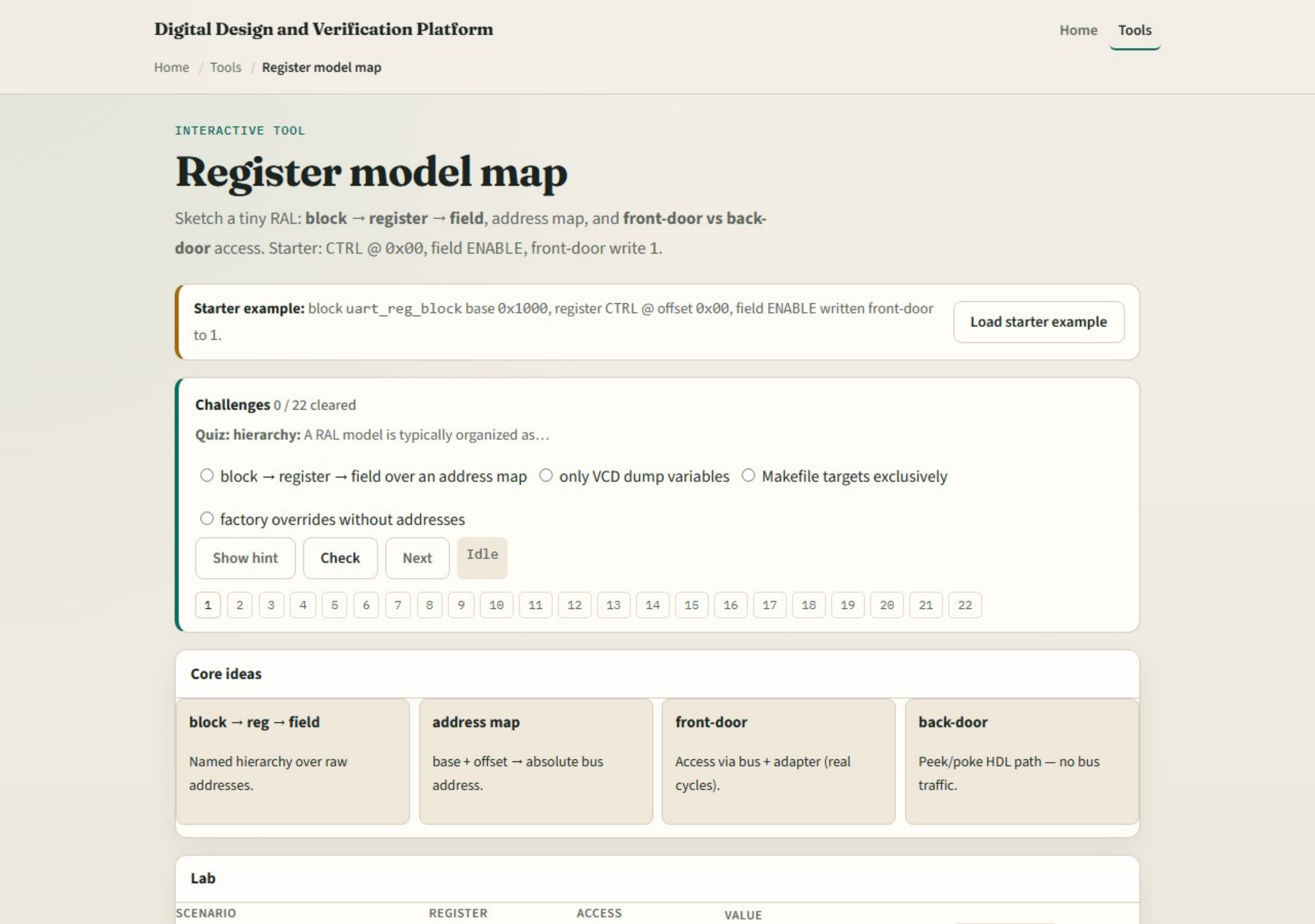
RAL map

Register tests often drown in raw hex addresses, RAL gives them names

Block, register, field, and access paths

- Think block contains registers
- Absolute address equals block base plus register offset
- Front-door access goes through the bus
- Back-door peek and poke uses an HDL path, no bus traffic, fast for debug and bring-up
- The mirror holds the model’s predicted value; compare mirror to DUT after reads
- Read-only fields block writes, that is access policy in the map

Browser lab



RAL map lab starter

Real UVM literacy

```
wsl — module14-ral-map/examples/ral-sketch

# Track A — real Unix

# real shell session (Track A — UVM RAL map)

$ pwd
/mnt/d/proj/designs/learning/courses/learn_uvm2017/module14-ral-map

$ ls examples/ral-sketch/
map.txt

$ cat examples/ral-sketch/map.txt
UVM RAL map sketch (literacy)
══════════════════════════════

Hierarchy:
  uvm_reg_block (uart_reg_block)
    base address = 0x1000
    ├── uvm_reg CTRL   @ offset 0x00  → abs 0x1000
    │     field ENABLE [0]   RW
    │     field RESERVED [7:1] RO
    ├── uvm_reg STATUS @ offset 0x04  → abs 0x1004
    │     field BUSY [0]     RO
    └── uvm_reg DATA   @ offset 0x08  → abs 0x1008
          field VALUE [7:0]  RW

Absolute address = block.base + reg.offset

Front-door (bus path):
  reg_model.uart_reg_block.CTRL.ENABLE.write(1, UVM_FRONTDOOR);
  // → reg adapter → bus driver → DUT register
  // mirror updated after successful transfer

Back-door (HDL path):
  reg_model.uart_reg_block.DATA.VALUE.poke(8'h5A, UVM_BACKDOOR);
  reg_model.uart_reg_block.STATUS.BUSY.peek(status, UVM_BACKDOOR);
  // no bus cycles — direct hierarchical reference

Mirror:
  predicted value in the reg model
  compare after read: mirror vs sampled DUT value

Access policy:
  RW — read and write allowed
  RO — read ok; write blocked
  WO — write only (less common in sketches)

Typical bring-up flow:
  1) build reg model from IP spec / reg spec
  2) lock model, connect adapter to bus agent
  3) reset + default map
  4) front-door for integration tests; back-door for debug

Common mistakes:
  raw address math when map has names
  backdoor poke without updating mirror expectations
  writing RO fields
  front-door without working adapter/reg predictor

$ grep -n "class uvm_reg_block\|uvm_reg_map\|frontdoor\|backdoor" ../../learn_uvm2017_sv_verilator/tools/uvm-
2017/1800.2-2017-1.0/src/reg/uvm_reg_block.svh | head -10
31:class uvm_reg_block extends uvm_object;
46:   local bit            maps[uvm_reg_map];
57:   local uvm_reg_backdoor backdoor;
129:   extern virtual function uvm_reg_map create_map(string name,
159:   extern function void set_default_map (uvm_reg_map map);
173:   uvm_reg_map default_map;
175:   extern function uvm_reg_map get_default_map ();
180:   /*local*/ extern function void add_map   (uvm_reg_map map);
192:   // <uvm_reg_map::get_reg_by_offset()> and
193:   // <uvm_reg_map::get_mem_by_offset()> methods.
```

Real shell — RAL map sketch

Pitfalls to watch

- Do not hand-compute addresses when the map already defines them, use named registers
- Do not poke backdoor and forget to update the mirror, or predict and DUT drift apart
- Do not write read-only fields and expect success
- Front-door needs a working adapter; back-door needs a valid HDL path
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, front-door write CTRL then explain absolute address
- On real UVM, sketch block, one register, and one field with base plus offset
- When you are ready